

DIA 3

LangGraph: orquestrando um agente

Trocando o loop por um grafo de estado, com memória, hooks e ponto de parada

1

Por que grafo e não for

2

Estado, nós, arestas,
reducers

3

Checkpoint, session
managers e context
engineering

4

Hooks, steering e
aprovação humana

As oito horas de hoje

1

09:00 – 10:30 · Por que grafo, e os quatro conceitos

Estado, nós, arestas e compilação. Ao fim do bloco, o loop de ontem redesenhado como grafo.

2

10:45 – 11:30 · Memória de verdade, e laboratório

Checkpoint, thread_id, session managers, poda de contexto. Depois, teclado.

3

13:15 – 14:45 · Constraints e capability

Hooks que negam, sandbox, steering, skills e MCP. O bloco que separa harness de prompt.

4

15:00 – 15:30 · Saindo da máquina e aprovação humana

Ferramentas de web, interrupt, Command, e laboratório do NEXUS com human-in-the-loop.

5

15:30 – 16:00 · Streaming, testes e fechamento

Como mostrar progresso e como testar roteamento sem carregar modelo.

01 Por que um grafo

O controle de fluxo preso dentro de uma função.

Cinco coisas que quebram o loop de ontem

1

Antes de salvar o arquivo, peça confirmação

Vira uma flag e um input() no meio do loop, e não sobrevive a um processo que morre.

2

Se a busca não achou nada, reformule uma vez

Vira um if aninhado com contador próprio, e mais um estado escondido em variável local.

3

Rode a análise de três documentos em paralelo

Vira threads dentro do loop, com resultados disputando a mesma variável.

4

O usuário fechou o notebook e volta amanhã

Vira serialização manual de estado, feita à mão e sempre incompleta.

5

Mostre na tela o que o agente está fazendo

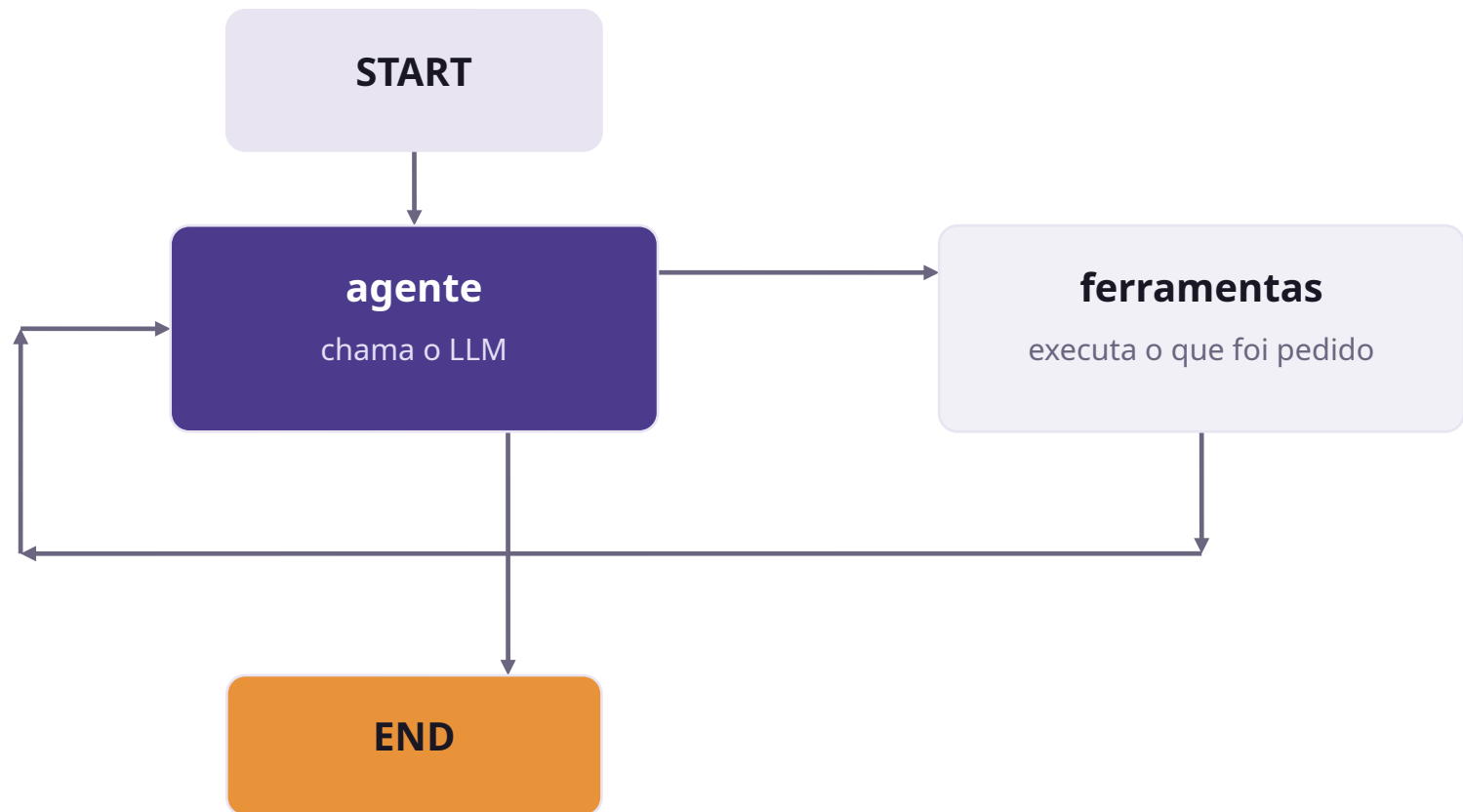
Vira print() espalhado por toda parte, misturado com a lógica.

A TROCA

Descreva o fluxo como nós e arestas, e deixe a biblioteca executá-lo.

O que era controle de fluxo implícito vira estrutura de dados explícita. E estrutura de dados você consegue inspecionar, desenhar, testar, versionar e persistir. O loop continua existindo: ele só não está mais escondido dentro de uma função sua.

O mesmo loop de ontem, desenhado



A conditional edge a partir de agente pergunta: tem tool calls? Vai para ferramentas. Não tem? Vai para END. O caminho de retorno leva o resultado de volta ao agente.

Por que StateGraph + checkpoint são essenciais para o harness

Harness manual

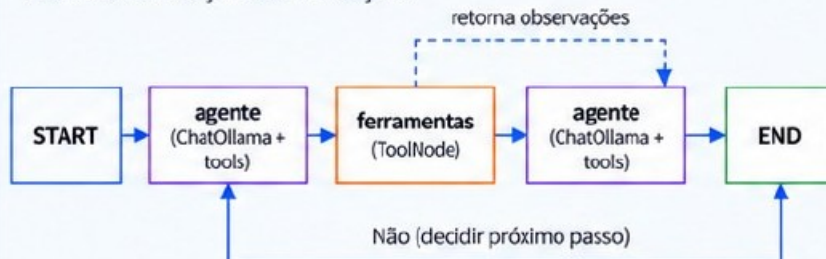
Um loop simples pode funcionar no início, mas tende a ficar difícil de retomar, depurar e auditar.



- O estado existe, mas fica implícito no código.
- Difícil retomar uma execução interrompida.
- Pouca visibilidade para auditoria e depuração.

StateGraph

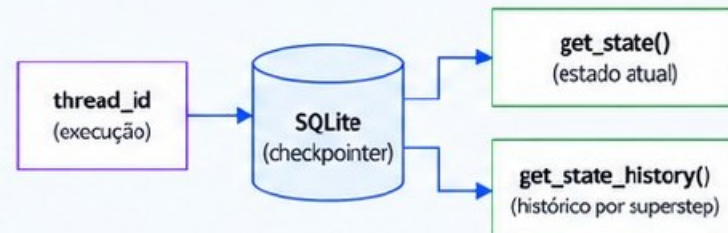
Modela o harness como uma máquina de estado, com nós, arestas e condições de transição.



- Estado explícito (mensagens, tool_calls, resultados, etc.).
- Transições controladas por arestas e condições.
- Pontos de interrupção bem definidos na execução.

Checkpoint

Persiste o estado em um banco (SQLite), permitindo retomar execuções e inspecionar o histórico.



- Retomar execução a partir de um checkpoint.
- Memória por thread (cada execução tem seu histórico).
- Auditoria por superstep (estado em cada passo).

Exemplo de código

```
1 conn = sqlite3.connect("nexus.db", check_same_thread=False)
2 saver = SqliteSaver(conn)
3 app = grafo.compile(checkpointer=saver)
4 app.invoke(entrada, {"configurable": {"thread_id": "aula-03"}})
```

O harness deixa de ser apenas um loop e passa a ser uma máquina de estado persistente, auditável e retomável.

O que a mudança de representação compra

Capacidade	Com for e if	Com grafo
Desenhar o fluxo	diagrama à mão que envelhece	gerado do próprio código
Pausar e retomar	serialização manual	checkpointer, de graça
Testar uma decisão isolada	impossível sem rodar tudo	função pura, testável
Ver o progresso	prints espalhados	stream por nó
Rodar ramos em paralelo	threads e variável compartilhada	arestas paralelas e reducer
Voltar a um passo anterior	não existe	histórico de checkpoints

Quando NÃO usar grafo

N Não precisa de grafo

- Uma chamada de modelo e pronto
- Sequência fixa sem ramificação: use LCEL
- Script que roda e morre, sem estado
- Protótipo de dez minutos

S Precisa de grafo

- Caminho depende do resultado do passo anterior
- Precisa pausar e retomar depois
- Estado sobrevive ao processo
- Vários agentes coordenados: dia 4

Três perguntas

1 Onde foi parar o for do dia 1?

Virou a aresta de retorno de ferramentas para agente.

2 Onde foi parar o if not tool_calls?

Virou a conditional edge que sai do nó agente.

3 Onde foi parar o max_passos?

Virou um campo do estado, lido pela função de roteamento.

02

Os quatro conceitos

Estado, nós, arestas, compilar.

Estado e reducers

O estado é um dicionário tipado que atravessa o grafo. Cada nó recebe o estado e devolve APENAS o que mudou.

O Annotated com add_messages define um reducer: como combinar o valor antigo com o novo.

Sem reducer, o valor novo SUBSTITUI o antigo. Com add_messages, a lista nova é ANEXADA, com deduplicação por id.

Esquecer o reducer em messages faz cada nó apagar o histórico inteiro. É o erro número um de quem começa.

```
from typing import Annotated, TypedDict
from langgraph.graph import add_messages

class Estado(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    pergunta: str        # substitui
    tentativas: int       # substitui

# outros reducers uteis:
import operator
fontes: Annotated[list[str], operator.add]
custo: Annotated[float, operator.add]

def juntar(antigo: dict, novo: dict) -> dict:
    return {**antigo, **novo}
achados: Annotated[dict, juntar]
```

O que acontece sem o reducer

X `messages: list[AnyMessage]`

- Cada nó SUBSTITUI a lista inteira
- O histórico some a cada passo
- O agente esquece o que a ferramenta devolveu
- Sintoma: loop infinito chamando a mesma coisa
- Não dá erro. Só funciona mal.

OK `Annotated[list, add_messages]`

- Cada nó ANEXA à lista
- Deduplicação por id de mensagem
- Atualização de mensagem existente pelo mesmo id
- Sintoma de sucesso: o histórico cresce
- É o comportamento que você espera

Nós, arestas e compilação

```
def no_agente(estado: Estado) -> dict:
    return {"messages": [llm_com_tools.invoke(estado["messages"])]}    # so o delta

construtor = StateGraph(Estado)
construtor.add_node("agente", no_agente)
construtor.add_node("ferramentas", ToolNode(FERRAMENTAS))

construtor.add_edge(START, "agente")          # aresta fixa
construtor.add_edge("ferramentas", "agente")  # o loop

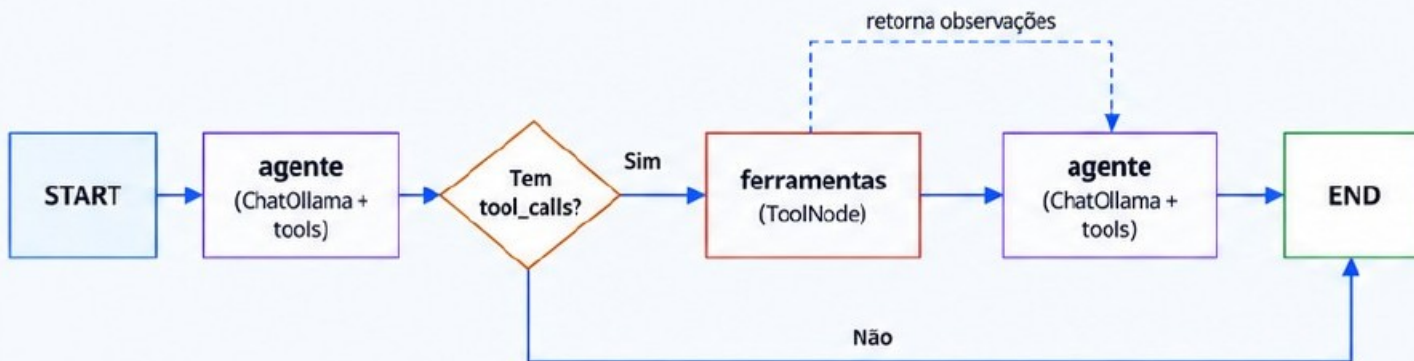
def rotear(estado: Estado) -> str:            # conditional edge
    ultima = estado["messages"][-1]
    return "ferramentas" if getattr(ultima, "tool_calls", None) else END

construtor.add_conditional_edges("agente", rotear, ["ferramentas", END])
grafo = construtor.compile()
```

O terceiro argumento de `add_conditional_edges` é a lista de destinos possíveis: opcional para executar, obrigatório para desenhar corretamente, e útil como documentação.

StateGraph na prática: nós, arestas e o ciclo agente → ferramentas

Fluxo do grafo



- O **estado** carrega mensagens e metadados da conversa (ex.: histórico, tool_calls, resultados).
- As **arestas** controlam a execução, definindo a sequência e as transições entre os nós.
- O ciclo só chama ferramentas quando há tool_calls na resposta do agente.

Exemplo de código

```
1 from langgraph.graph import StateGraph, START, END
2 from langgraph.prebuilt import ToolNode
3
4 grafo = StateGraph(Estado)
5 grafo.add_node("agente", chamar_modelo)
6 grafo.add_node("ferramentas", ToolNode(ferramentas))
7
8 grafo.add_edge(START, "agente")
9 grafo.add_conditional_edges("agente", decidir_proximo)
10 grafo.add_edge("ferramentas", "agente")
11
12 app = grafo.compile()
```

Desenhe antes de rodar

Desenhar o grafo antes de executar pega metade dos erros de arquitetura: nó órfão, aresta faltando, ciclo indesejado, destino que não existe.

Peça que o diagrama gerado pelo próprio código entre no README do projeto. Diagrama desenhado à mão envelhece; diagrama gerado, não.

```
print(grafo.get_graph().draw_ascii())

png = grafo.get_graph().draw_mermaid_png()
open("grafo.png", "wb").write(png)
```

```
#      +-----+
#      |  START  |
#      +-----+
#              *
#      +-----+
#      |  agente  |
#      +-----+
#              .
# +-----+ +-----+
# | ferramentas | | END |
# +-----+ +-----+
```

Escrever à mão ou usar o pronto

1 À mão: didático

- `for tc in estado['messages'][-1].tool_calls`
- Busca a ferramenta no REGISTRO
- Trata ferramenta inexistente
- Captura exceção e vira texto de erro
- Devolve lista de ToolMessage
- Escreva assim em sala, para todos verem

2 ToolNode: produção

- `add_node('ferramentas', ToolNode(FERRAMENTAS))`
- Executa tool calls em paralelo quando o modelo pede várias
- Injeta ToolRuntime automaticamente
- Tratamento de erro configurável
- `tools_condition` faz o roteamento pronto
- Troque depois que todos entenderem

O grafo inteiro, em uma tela

```
class Estado(TypedDict):
    messages: Annotated[list[AnyMessage], add_messages]
    passos: int

def no_agente(estado: Estado) -> dict:
    ai = llm_com_tools.invoke([SystemMessage(SISTEMA)] + estado["messages"])
    return {"messages": [ai], "passos": estado.get("passos", 0) + 1}

def rotear(estado: Estado) -> str:
    ultima = estado["messages"][-1]
    if estado.get("passos", 0) >= 10:                # parada por limite
        return END
    return "ferramentas" if getattr(ultima, "tool_calls", None) else END

g = StateGraph(Estado)
g.add_node("agente", no_agente)
g.add_node("ferramentas", ToolNode(FERRAMENTAS))
g.add_edge(START, "agente")
g.add_edge("ferramentas", "agente")
g.add_conditional_edges("agente", rotear, ["ferramentas", END])
grafo = g.compile()
```

Vinte linhas. É o agente do dia 1 inteiro, agora com o limite de passos como dado do estado em vez de variável de loop.

Três perguntas antes do intervalo

1 Um nó devolveu o estado inteiro e um campo sumiu

Por quê?

2 O agente repete a mesma ferramenta para sempre

Qual é o primeiro suspeito?

3 Você quer somar o custo de cada nó

O que precisa no estado?

03

Memória e sessão

O ganho que sozinho justifica a mudança.

Memória sem escrever código de memória

O checkpointer salva o estado inteiro depois de cada nó, indexado por thread_id.

Consequências diretas: conversa com memória sem código de memória; retomada exata depois de queda do processo; várias conversas independentes no mesmo grafo; e viagem no tempo, voltando a um checkpoint e seguindo por outro caminho.

```
from langgraph.checkpoint.sqlite import SqliteSaver

with SqliteSaver.from_conn_string("nexus.db") as mem:
    grafo = construtor.compile(checkpointer=mem)

    cfg = {"configurable": {"thread_id": "ana"}}

    grafo.invoke({"messages":
        [HumanMessage("Meu nome e Ana.")]}], cfg)

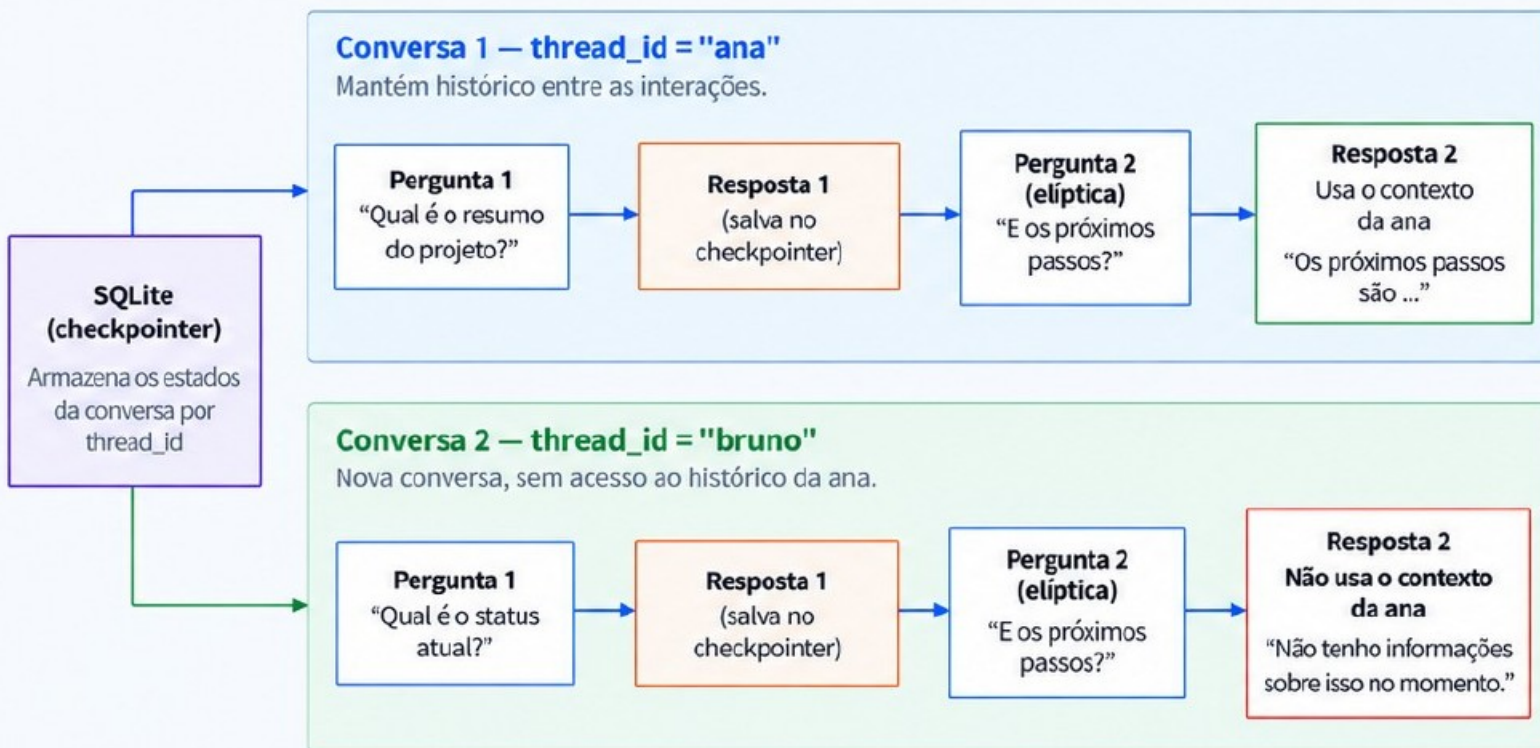
    r = grafo.invoke({"messages":
        [HumanMessage("Qual e o meu nome?")]}], cfg)
    print(r["messages"][-1].content)    # Ana

    # thread diferente = memoria diferente
    cfg2 = {"configurable": {"thread_id": "bruno"}}
```

Memória por thread_id: contexto isolado por conversa

Cada thread_id mantém seu próprio histórico no checkpointer, isolando o contexto entre conversas diferentes.

Duas conversas em paralelo, com contextos isolados



Exemplo de código

```
1 # Conversa da ana
2 cfg_ana = {"configurable": {"thread_id": "ana"}}
3 res1 = app.invoke(
4     {"messages": [{"role": "user",
5                     "content": "Qual é o resumo do projeto?"}]},
6     cfg_ana
7 )
8
9 # Conversa do bruno (nova thread, contexto limpo)
10 cfg_bruno = {"configurable": {"thread_id": "bruno"}}
11 res2 = app.invoke(
12     {"messages": [{"role": "user",
13                     "content": "Qual é o status atual?"}]},
14     cfg_bruno
15 )
```

Principais pontos

- 1 Mesma thread mantém histórico**
O contexto da conversa é preservado entre as chamadas.
- 2 Nova thread isola o contexto**
Cada thread_id começa uma conversa limpa e independente.
- 3 Memória depende do checkpointer**
Os estados são persistidos no banco SQLite via checkpointer.

Ler e navegar o estado salvo

```
estado = grafo.get_state(config)
print(estados.values["messages"][-1])      # o que esta no estado agora
print(estados.next)                        # qual no vai executar em seguida

for snap in grafo.get_state_history(config):
    print(snap.config["configurable"]["checkpoint_id"], snap.next)

# retomar de um checkpoint anterior e seguir por outro caminho:
anterior = list(grafo.get_state_history(config))[3]
grafo.invoke(None, anterior.config)
```

estados.next vazio significa que o grafo terminou. Não vazio significa que ele está pausado, e é assim que a aprovação humana funciona.

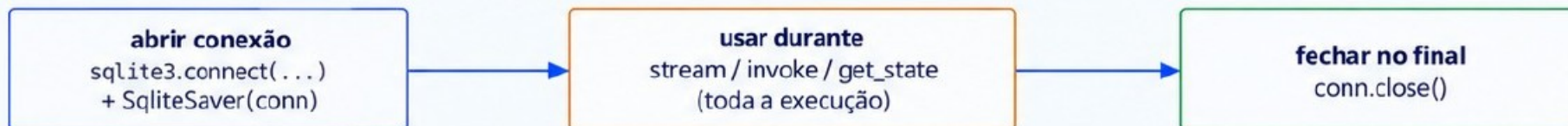
Lifecycle correto do SQLite: abrir, usar e fechar no momento certo

Correto

```
1 import sqlite3
2 from langgraph.checkpoint.sqlite import SqliteSaver
3
4 conn = sqlite3.connect("nexus.db", check_same_thread=False)
5 saver = SqliteSaver(conn)
6 app = grafo.compile(checkpointer=saver)
7
8 # usar normalmente (stream, invoke, get_state, etc.)
9 resultado = app.invoke({"mensagem": "Olá!"})
10
11 # fechar somente ao final
12 conn.close()
```

Incorreto

```
1 import sqlite3
2 from langgraph.checkpoint.sqlite import SqliteSaver
3
4 with sqlite3.connect("nexus.db") as conn:
5     saver = SqliteSaver(conn)
6     app = grafo.compile(checkpointer=saver)
7
8 # ao sair do bloco, a conexão é fechada
9 # mas o grafo pode continuar usando o checkpointer
10 # causando erros intermitentes
```



- Mantenha a conexão viva durante toda a vida do grafo (stream, invoke, get_state, etc.).
- Evite erros intermitentes causados por conexão fechada prematuramente.
- Separe a criação e o gerenciamento dos recursos (conexão, saver) do uso do grafo.

Quem é o dono da memória

1

thread_id não é id de conversa: é chave de acesso

Quem manda o thread_id lê o histórico daquele thread. Se ele vier do cliente sem validação, qualquer um lê a conversa de qualquer um.

2

A chave carrega o dono

Monte sempre como uma composição do usuário autenticado com o identificador da conversa. Nunca aceite o thread_id cru vindo do navegador.

3

Uma sessão, um escopo

Conversas diferentes do mesmo usuário são threads diferentes. Misturar assuntos num thread só é a receita para contexto estourado e resposta confusa.

4

Expiração e limpeza são requisito

Defina há quanto tempo uma sessão morre e o que acontece com o histórico. Se ninguém decidir, a decisão vira crescer para sempre.

Qual backend de checkpoint

Backend	Quando usar	Cuidado
MemorySaver	Teste automatizado e experimento	Some quando o processo morre
SqliteSaver	Curso, protótipo, ferramenta local	Um saver por processo; trava com concorrência
PostgresSaver	Produção com vários processos	Precisa de migração inicial da tabela
Implementação própria	Requisito de retenção ou criptografia	Você assume a correteude do reducer

Memória infinita não é bênção

1

Depois de trinta turnos, o contexto estoura

Com modelo local de 8k a 16k isso acontece mais cedo do que se imagina, e o sintoma é uma resposta subitamente ruim, não um erro.

2

Antes de estourar, já piorou

É o context rot do dia 1: a recuperação degrada bem antes do limite. Contexto cheio é contexto pior, não só contexto caro.

3

Janela deslizando

Um nó que mantém o system prompt e as últimas N mensagens, removendo o resto com RemoveMessage.

4

Sumarização

Ao passar de N mensagens, pedir ao modelo um resumo das antigas e substituí-las por ele. Guarde o resumo no estado, não no histórico.

Poda de contexto como nó do grafo

```
from langchain_core.messages import RemoveMessage

MANTER = 12          # ultimas mensagens preservadas

def no_podar(estados: Estado) -> dict:
    msgs = estados["messages"]
    if len(msgs) <= MANTER:
        return {}
    antigas = msgs[:-MANTER]
    resumo = llm.invoke(
        [SystemMessage("Resuma em ate 8 linhas o que foi decidido e descoberto.")]
        + antigas).content
    return {"resumo": resumo,
            "messages": [RemoveMessage(id=m.id) for m in antigas]}

g.add_node("podar", no_podar)
g.add_edge("podar", "agente")
```

A política de contexto vira um nó visível, desenhável e testável, e não um if escondido dentro do loop.

Laboratório da manhã: o NEXUS vira grafo

1

Migrar o agente de ontem para StateGraph

Dois nós, uma aresta fixa, uma condicional. Rode o `draw_ascii` e confira o desenho.

2

Trazer as quatro condições de parada

Como campos do estado e ifs no rotear. Nenhuma delas vem do framework.

3

Ligar o SqliteSaver e provar a memória

Duas perguntas no mesmo `thread_id`, e a terceira em outro thread para provar o isolamento.

4

Montar o `thread_id` com o dono

Componha usuário e conversa. Escreva um teste que garante que dois usuários não leem o mesmo histórico.

5

Extensão: o nó de poda

Para quem terminar antes. Meça o tamanho do contexto antes e depois.

04

Constraints de verdade

Onde o harness deixa de ser arte e vira engenharia.

Sugestão e restrição não são a mesma coisa

S Sugestão (prompt)

- Nunca faça push direto na main
- Garantia probabilística
- Degrada com contexto longo
- Custa tokens em toda chamada
- O agente pode contornar

R Restrição (hook)

- PreToolUse intercepta e nega
- Garantia determinística
- Não degrada nunca
- Custa zero token
- O agente não contorna

Onde enganchar, e qual deles pode negar

Evento	Quando dispara	Para que serve
PreToolUse	Antes de executar	NEGAR a operação: o ponto central de segurança
PostToolUse	Depois de executar	Auditoria, log, formatação automática
SessionStart	Início da sessão	Carregar contexto, inicializar ambiente
Stop	O agente decidiu parar	Verificação determinística de conclusão
PermissionRequest	Pediu permissão	Aprovação automática, ou rota para um humano
PostCompact	Depois de compactar	Reagir à perda de contexto

Políticas como funções pequenas e testáveis

Não precisamos do Claude Code para ter isso. O nosso ToolNode já é o ponto de interceptação: falta transformá-lo num lugar onde políticas se registram.

Duas coisas: estas políticas são testáveis SEM LLM nenhum, e vão para o pytest junto com o roteamento. E a mensagem de recusa volta ao modelo como observação, então ele aprende no meio da execução que aquele caminho está bloqueado.

```
PRE = []

def pre_tool_use(fn):
    """Devolver string = NEGAR com esse motivo."""
    PRE.append(fn); return fn

def executar_com_hooks(ferramenta, args):
    for politica in PRE:
        recusa = politica(ferramenta, args)
        if recusa:
            return f"NEGADO pela politica: {recusa}"
    return REGISTRO[ferramenta].invoke(args)

@pre_tool_use
def proteger_fora_da_pasta(ferramenta, args):
    if ferramenta == "salvar_relatorio":
        destino = Path(args["caminho"]).resolve()
        if not str(destino).startswith(str(SAIDA)):
            return f"escrita fora de {SAIDA} nao permitida"
```

DETALHE QUE FAZ DIFERENÇA

Uma recusa bem escrita é uma instrução.

Comparem: NEGADO contra NEGADO: escrita fora de saida/ não é permitida; use saida/relatorio.md. A primeira faz o agente tentar de novo até o limite de passos. A segunda faz ele acertar na volta seguinte. É o mesmo princípio do erro útil do dia 1, aplicado à camada de constraints.

Hooks bloqueiam o que você previu. Sandbox, o resto

Nível	Mecanismo	Bloqueia
Caminho	resolve() mais checagem de prefixo	Escapar da pasta do projeto
Processo	Usuário sem privilégio, ulimit	Consumo desenfreado de CPU e memória
Rede	Desligada por padrão	Exfiltração e chamadas não previstas
Sistema	Container	Todo o resto

Manter o agente nos trilhos durante a execução

1

Steering é correção em tempo de execução

Não é reescrever o prompt e rodar de novo: é intervir enquanto o agente trabalha, com base no que ele já fez.

2

Injetar contexto no meio

Um nó que percebe desvio e acrescenta uma mensagem: você já tentou isso duas vezes, mude de abordagem.

3

Cortar caminho

Roteamento forçado quando o estado mostra que o caminho atual não vai dar certo. É o Command com goto.

4

Intervenção humana

O usuário manda uma correção no meio da execução, e ela entra como mensagem no próximo passo. É human-in-the-loop contínuo, não só aprovação.

O ACHADO CONTRAINTUITIVO

Aumentar a autonomia da IA exige diminuir a flexibilidade do espaço de solução.

Encolher o espaço de busca melhora a saída do agente. Um agente que pode fazer qualquer coisa erra mais que um que pode fazer cinco coisas bem definidas. É o oposto do que a intuição sugere, e está medido no caso do Codex.

Classifique cada regra

1

Escreva no tom formal da empresa

Instruction, constraint ou steering?

2

Nunca apague arquivo fora de tmp/

Instruction, constraint ou steering?

3

Se tentou duas vezes e falhou, mude de abordagem

Instruction, constraint ou steering?

05

Capability em escala

Skills, plugins, MCP, e o custo de ter ferramenta demais.

A TENSÃO

Cada descrição de ferramenta disputa espaço com o trabalho de verdade.

Quanto mais ferramentas você conecta, mais o agente consegue fazer. Mas contexto é finito: vinte ferramentas conectadas são vinte descrições ocupando a janela antes de a tarefa começar. Capability não é acumular ferramenta: é decidir o que carregar e quando.

Capacidade que só carrega quando precisa

Uma skill é um arquivo Markdown que descreve o que a capacidade faz e quando usá-la. Não ocupa contexto por padrão: o agente lê as descrições, decide se é relevante, e só então carrega o conteúdo.

Escrever um arquivo .md define uma capacidade nova. Sem código, sem SDK. É o mecanismo de extensão de menor barreira que existe hoje.

```
.claude/skills/  
  publicar-relatorio/SKILL.md  
  conferir-numeros/SKILL.md  
  resumir-reuniao/SKILL.md  
  
# dentro de SKILL.md:  
---  
name: conferir-numeros  
description: >  
  Confere os numeros de um relatorio contra  
  as fontes citadas. Use antes de publicar  
  qualquer documento com dados.  
---  
  
(o conteudo so entra no contexto quando  
a descricao acima casa com a tarefa)
```

Quando a extensão é um pacote, e não um arquivo

1

Plugin agrupa o que anda junto

Skills, ferramentas, configuração de servidores MCP e hooks, distribuídos como uma unidade instalável.

2

Serve para padronizar time

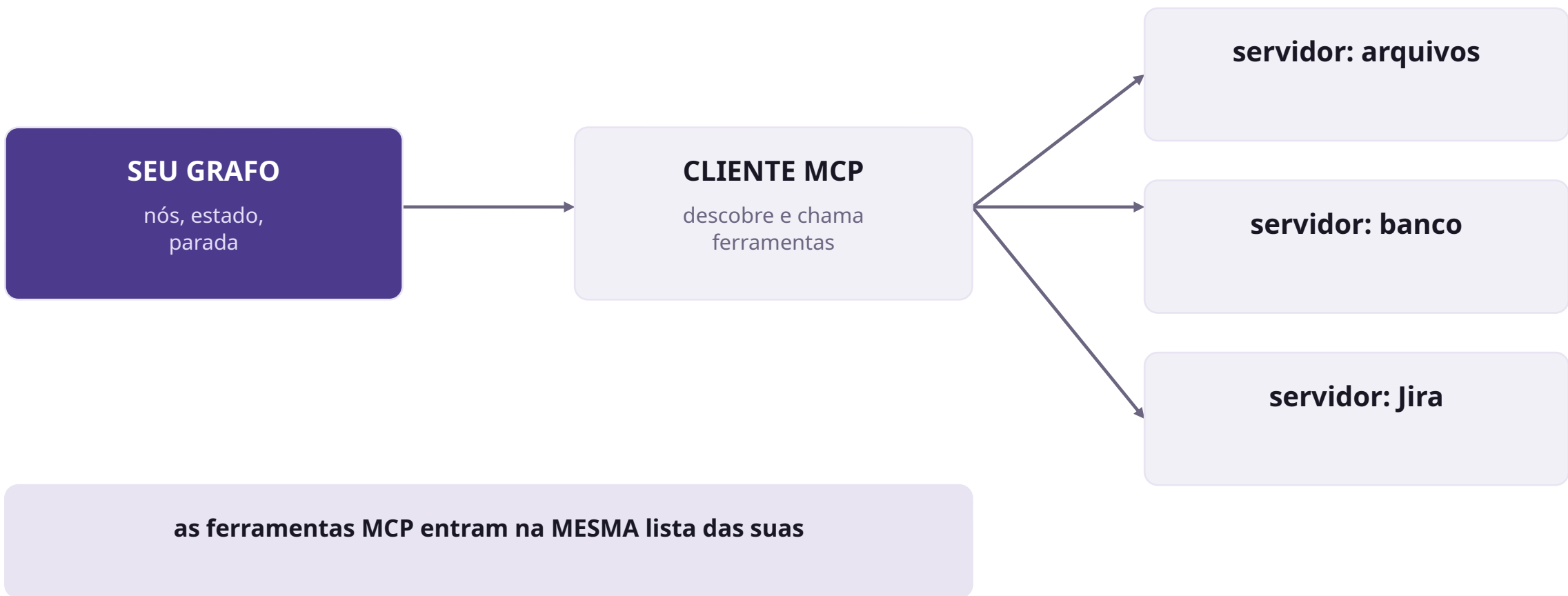
Em vez de cada pessoa configurar do seu jeito, a equipe instala o mesmo pacote e todos têm o mesmo harness.

3

O ganho é governança, não capacidade

Nada que um plugin faz é impossível de fazer à mão. O que ele traz é versão, distribuição e revisão.

O protocolo que padroniza como a ferramenta se apresenta



O MCP não decide o loop, a parada nem a política de contexto. Ele resolve só como um servidor descreve o que oferece.

Ferramentas de fora entram na MESMA lista

Skills são capacidade local. Quando o agente precisa falar com o mundo, banco, API, repositório, sistema de tickets, entra o Model Context Protocol.

A linha que importa é a última. Do ponto de vista do loop que você escreveu no dia 1, não há diferença: é um nome, um schema e uma função que devolve texto.

```
from langchain_mcp_adapters.client import (
    MultiServerMCPClient)

cliente = MultiServerMCPClient({
    "arquivos": {
        "command": "npx",
        "args": ["-y",
                 "@modelcontextprotocol/"
                 "server-filesystem", "./docs"],
        "transport": "stdio"},
})

ferramentas_mcp = await cliente.get_tools()

FERRAMENTAS = [calcular,
                buscar_documentos] + ferramentas_mcp
```

Para o agente, a descrição da ferramenta É a interface

Faça	Não faça
Nome que revela intenção: buscar_base_de_conhecimento	Nome vago: processar_dados
Cada parâmetro com descrição e exemplo	Parâmetro sem descrição, o agente adivinha
Uma ferramenta faz uma coisa e devolve	Uma faz cinco coisas e tem efeito colateral
Erro explica o que houve e o que fazer	Erro diz apenas que falhou

FECHO DO BLOCO

**Instructions dizem. Constraints impedem.
Capability dá o corpo. Steering corrige no meio.**

Os quatro trabalham juntos e nenhum substitui o outro. O erro mais caro do campo é tentar resolver com prompt o que precisava de código, e o segundo mais caro é conectar cinquenta ferramentas e chamar isso de arquitetura.

Padrões também usados por agentes reais

LangGraph

- estado explícito (grafo de nós/arestas)
- roteamento condicional
- checkpoint e retomada



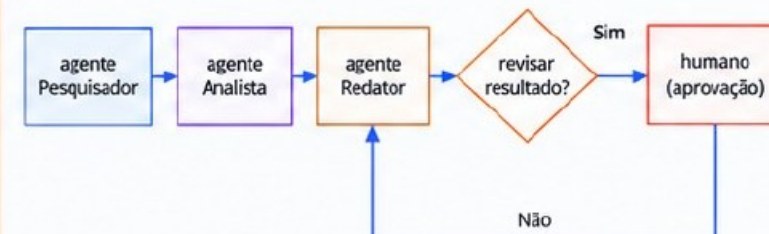
AutoGen

- múltiplos agentes em conversa
- contexto compartilhado (memória)
- gate humano quando necessário



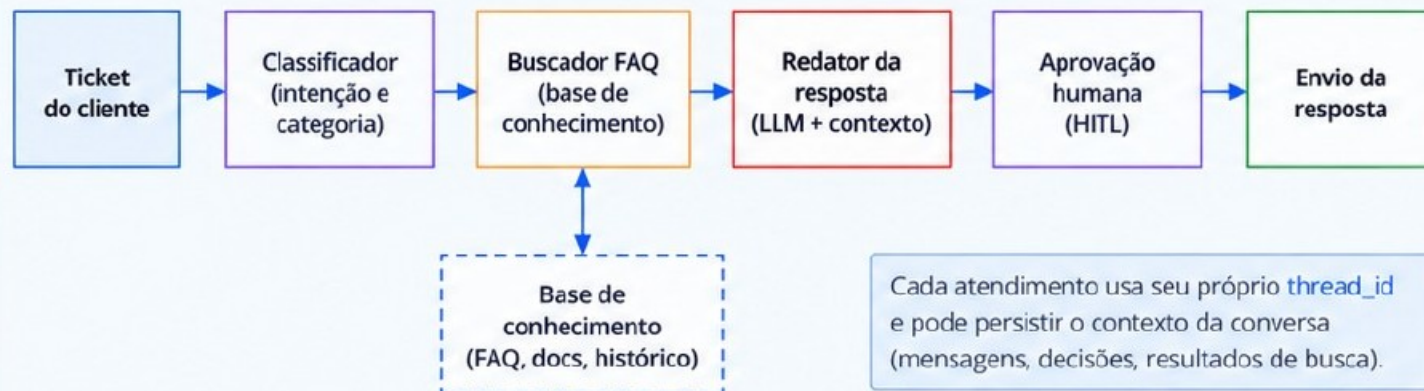
CrewAI

- papéis especializados (múltiplos agentes)
- tarefas encadeadas (pipeline)
- intervenção humana quando necessário



Exemplo real 1: agente de suporte com memória e aprovação

Arquitetura do fluxo



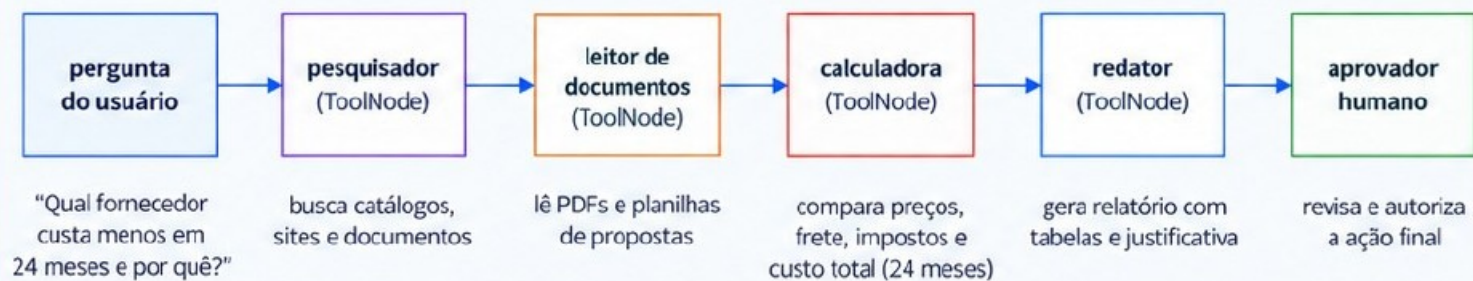
- A memória permite entender perguntas de acompanhamento, usando o histórico do atendimento.
- O HITL reduz o risco de respostas incorretas ou inadequadas chegarem ao cliente.
- O grafo organiza o processo em nós e arestas, facilitando auditoria, métricas e depuração.

Exemplo de código (pseudo-código)

```
1 # fluxo simplificado de um agente de suporte
2 def processar_ticket(ticket, thread_id):
3     categoria = classificar(ticket)
4     contexto = recuperar_contexto(
5         thread_id=thread_id,
6         categoria=categoria
7     )
8     resposta = redigir_resposta(
9         ticket=ticket,
10        contexto=contexto
11    )
12    aprovada = aprovar_antes_de_enviar(
13        resposta=resposta,
14        thread_id=thread_id
15    )
16    if aprovada:
17        enviar_resposta(ticket, resposta)
18        persistir_mensagem(thread_id, "resposta_enviada", resposta)
19    else:
20        persistir_mensagem(thread_id, "ajuste_solicitado", resposta)
```

Exemplo real 2: agente de compras e comparação de fornecedores

Arquitetura do agente



- O grafo encadeia raciocínio com ferramentas, onde cada nó executa uma etapa específica da tarefa.
- A aprovação humana protege a ação final (ex.: salvar e compartilhar o relatório de comparação).
- O estado explícito permite rastrear fontes, dados extraídos e cálculos, facilitando auditoria e explicação.

Exemplo de código

```
1 # Trecho simplificado de ferramentas usadas pelo agente
2 arq = ler_arquivo("proposta_fornecedor_A.pdf")
3 dados_a = extrair_preços(arq)
4 arq = ler_arquivo("proposta_fornecedor_B.pdf")
5 dados_b = extrair_preços(arq)
6 resultado = calcular(
7     dado_a=dados_a,
8     dado_b=dados_b,
9     horizonte_meses=24,
10    considerar_frete=True,
11 )
12 relatorio = salvar_relatorio(resultado, "comparacao_fornecedores.md")
```

06

Saindo da máquina

Até agora o agente só leu o próprio disco.

A TENSÃO QUE NINGUÉM NOMEIA

O modelo continua local. A informação, não.

Buscar na web significa que a PERGUNTA do usuário sai da máquina. Não os pesos, não o histórico, mas a consulta. Para um agente de pesquisa pública isso é irrelevante. Para um que processa dado de paciente ou cláusula de contrato, é inaceitável. A decisão é do projeto, não do framework.

Busca que funciona para vinte alunos ao mesmo tempo

Nenhuma chave, nenhuma conta. É a escolha certa para sala de aula, e o Tavily entra como demonstração de três minutos.

Repare que a ferramenta devolve a URL junto com o trecho. Sem isso o Redator não cita fonte e o Crítico não verifica nada.

```
from ddgs import DDGS
from langchain_core.tools import tool

@tool
def buscar_web(consulta: str, k: int = 5) -> str:
    """Busca na web: título, trecho e URL.

    Use quando a resposta nao estiver nos
    documentos locais.
    """
    k = max(1, min(k, 10))
    try:
        with DDGS() as ddgs:
            rs = list(ddgs.text(consulta, max_results=k))
    except Exception as e:
        return f"ERRO na busca: {type(e).__name__}: {e}"

    if not rs:
        return "Nenhum resultado encontrado."
    return "\n\n---\n\n".join(
        f"[{r['title']}] \n {r['body']} \n Fonte: {r['href']}"
        for r in rs
    )
```

ler_pagina: HTML vira texto útil

A busca devolve um trecho. Às vezes você precisa da página inteira: a tabela de preços, o texto completo da norma.

Três detalhes que o aluno esquece: decompose antes de get_text, timeout sempre, e max_chars. Cada um resolve uma forma diferente de derrubar o agente.

```
CABECALHO = {"User-Agent": "NEXUS-curso/1.0"}

@tool
def ler_pagina(url: str, max_chars: int = 4000) -> str:
    """Baixa a pagina e devolve o texto sem HTML."""
    if not url.startswith(("http://", "https://")):
        return "ERRO: url invalida"
    try:
        r = requests.get(url, headers=CABECALHO,
                        timeout=15)
        r.raise_for_status()
    except Exception as e:
        return f"ERRO ao baixar: {type(e).__name__}"

    sopa = BeautifulSoup(r.text, "html.parser")
    for lixo in sopa(["script", "style", "nav",
                    "header", "footer", "aside"]):
        lixo.decompose()

    linhas = [l.strip() for l
              in sopa.get_text("\n").splitlines()
              if l.strip()]
    limpo = "\n".join(linhas)
    return limpo[:max_chars]
```

Quando a página só existe depois do JavaScript

Se a página aparece no navegador mas o requests devolve uma casca vazia, o conteúdo é renderizado no cliente. Aí não há como escapar de um navegador de verdade.

O finally com quit() não é preciosismo: sem ele, cada falha deixa um Chrome vivo. Vinte perguntas depois são dois gigabytes de navegadores fantasma.

```
@tool
def ler_pagina_dinamica(url: str, espera: int = 3,
                        max_chars: int = 4000) -> str:
    """Como ler_pagina, mas executa o JavaScript.

    Use so quando ler_pagina voltar vazio:
    e segundos, nao milissegundos.
    """
    opcoes = Options()
    opcoes.add_argument("--headless=new")
    opcoes.add_argument("--no-sandbox")

    navegador = None
    try:
        navegador = webdriver.Chrome(options=opcoes)
        navegador.get(url)
        time.sleep(espera)
        html = navegador.page_source
    except Exception as e:
        return f"ERRO no navegador: {e}"
    finally:
        if navegador is not None:
            navegador.quit()

    return _limpar_html(html, max_chars)
```

Quatro ferramentas parecidas exigem ordem explícita

Situação	Ferramenta	Custo típico
A resposta está nos documentos locais	buscar_documentos	milissegundos
Preciso de informação pública e atual	buscar_web	1 a 3 s
Preciso do texto completo de uma URL	ler_pagina	0,5 a 2 s
A página só carrega com JavaScript	ler_pagina_dinamica	5 a 15 s

O QUE ESTE BLOCO COMPROU

Três capacidades novas. Zero mudança no grafo.

As ferramentas entram na mesma lista, o ToolNode as executa igual, a topologia continua idêntica. Esse desacoplamento entre o que o agente PODE FAZER e como ele DECIDE é o que faz o LangGraph valer o custo de aprendê-lo.

07

Aprovação humana

Salvar arquivo sem confirmação é dar sudo ao modelo.

O que exige aprovação humana

1 Irreversível

Apagar, sobrescrever, enviar mensagem, publicar. Se desfazer é caro ou impossível, pare e pergunte.

2 Custa dinheiro

Compra, chamada de API cara, provisionamento de recurso. O agente não tem noção de orçamento.

3 Sai da organização

E-mail para cliente, post público, commit em repositório de terceiro. O erro vira dano de reputação.

Parar antes de um nó

A forma mais simples: declarar na compilação que o grafo deve parar antes de determinado nó.

`invoke(None, config)` é o idioma de retomada: sem entrada nova, continue de onde parou.

O processo pode morrer entre o pedido e a resposta. A retomada funciona porque o estado está no banco, e não na memória.

```
grafo = construtor.compile(
    checkpointer=memoria,
    interrupt_before=["ferramentas"],
)

grafo.invoke(entrada, config)      # para antes

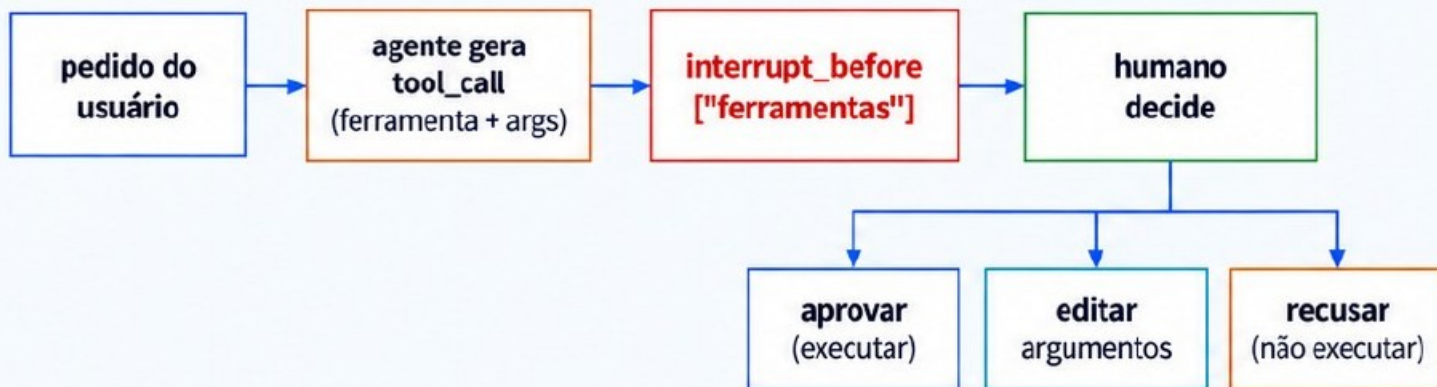
estado = grafo.get_state(config)
chamadas = estado.values["messages"][-1].tool_calls
print("vai executar:", chamadas)

if input("aprovar? [s/n] ") == "s":
    grafo.invoke(None, config)      # continua
```

HITL: aprovar, editar ou recusar antes da ferramenta

O humano no loop revisa, ajusta ou bloqueia a chamada de ferramentas antes da execução.

Fluxo com HITL antes das ferramentas



- O HITL protege ações sensíveis, exigindo validação humana antes de executar ferramentas.
- A execução é pausada logo antes da chamada da ferramenta, após o agente gerar o tool_call.
- Permite governança e auditoria sobre quais ferramentas são usadas e com quais argumentos.

Exemplo de código (Python)

```
# Compila o grafo com interrupção antes das ferramentas
app = grafo.compile(
    checkpointer=saver,
    interrupt_before=["ferramentas"]
)

# Executa e pausa no ponto de interrupção
estado = app.invoke(entrada, cfg)

# O estado retornado permite ao humano aprovar, editar
# os argumentos ou recusar a chamada da ferramenta
```

interrupt: pausar de dentro de um nó

Mais expressivo: o nó decide quando pausar e devolve dados para quem vai decidir.

O interrupt congela o grafo no checkpoint e devolve o payload para o chamador.

O retorno de interrupt() é exatamente o valor passado em Command(resume=...). Trate a recusa devolvendo um ToolMessage que explique o motivo: o modelo precisa saber que foi recusado, e por quê.

```
from langgraph.types import interrupt, Command

def no_aprovacao(estado: Estado) -> dict:
    ch = estado["messages"][-1].tool_calls[0]
    decisao = interrupt({
        "pergunta": "Posso executar esta acao?",
        "ferramenta": ch["name"],
        "argumentos": ch["args"],
    })
    if decisao == "nao":
        return {"messages": [ToolMessage(
            content="Usuario recusou. Explique e "
            "proponha alternativa.",
            tool_call_id=ch["id"])]}]
    return {}

# retomando:
grafo.invoke(Command(resume="sim"), config)
```

O ERRO QUE METADE DAS TURMAS COMETE

Implementar o sim e esquecer o não.

Quando o usuário recusa, o agente precisa saber que foi recusado, por quê, e o que fazer em seguida. Se você simplesmente não executa a ferramenta e devolve o controle, o modelo vê uma tool call sem resposta, fica confuso e tenta de novo. A recusa precisa virar uma observação no contexto, exatamente como um erro de ferramenta.

Command: as duas coisas de uma vez

Quando um nó devolve Command, ele não precisa de conditional edge: o roteamento vem junto com a atualização do estado.

Esta é a base do handoff entre agentes, que é o assunto de amanhã. Guarde o nome: Command com goto.

```
from langgraph.types import Command
from typing import Literal

def no_verificador(estado: Estado) -> Command[
    Literal["agente", "redator"]]:

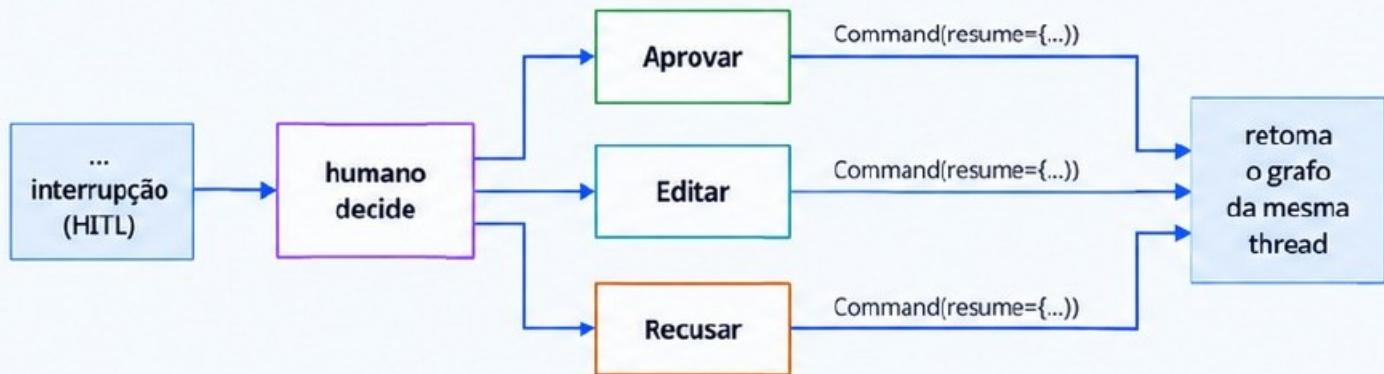
    if estado["tentativas"] >= 3:
        return Command(
            update={"aviso": "limite atingido"},
            goto="redator")

    return Command(
        update={"tentativas": estado["tentativas"] + 1},
        goto="agente")
```

Retomada após HITL: Command(resume=...) na prática

Após a interrupção, o humano decide e a execução continua a partir do mesmo ponto no grafo.

Retorno ao grafo após a decisão humana



- **get_state** inspeciona o ponto de pausa (estado, mensagens, tool_calls, etc.).
- **Command(resume=...)** retoma a execução a partir do mesmo ponto no grafo.
- A rejeição também é uma decisão válida e segue o fluxo definido pelo grafo.

Exemplo de código (Python)

```
from langgraph.types import Command

# Inspecciona o estado no ponto de pausa
snapshot = app.get_state(cfg)

# Retoma a execução com aprovação
app.invoke(Command(resume={"acao": "aprovar"}), cfg)

# Retoma editando argumentos
app.invoke(Command(
    resume={
        "acao": "editar",
        "novos_argumentos": {"cliente_id": 123, "limite": 1000}
    },
    cfg))
```

Laboratório da tarde: o NEXUS com freio

1

Adicionar salvar_relatorio como ferramenta

Escreve em saida/. É a primeira ação irreversível do curso.

2

Registrar duas políticas de PreToolUse

Uma que impede escrita fora de saida/ e outra à sua escolha. Com teste, sem LLM.

3

Pausar antes de salvar e pedir aprovação

Com interrupt, mostrando ao humano exatamente o que será escrito.

4

Implementar o caminho do não

A recusa vira ToolMessage com motivo, e o agente propõe alternativa.

5

Provar a retomada após queda

Pause, mate o processo, suba de novo e retome. Anote o que foi preciso para funcionar.

Laboratório guiado do notebook: do stream ao HITL

1 executar stream do grafo

Inicie o grafo em modo de streaming e observe os eventos e tool_calls.

2 repetir com a mesma thread_id

Execute novamente a mesma pergunta usando a mesma thread_id.

3 comparar com thread_id nova

Execute a mesma pergunta, agora com uma thread_id diferente.

4 ativar interrupt_before

Habilite interrupt_before=["ferramentas"] e execute uma ação sensível.

5 aprovar e depois recusar uma ação sensível

Use o Command para aprovar a execução e, em seguida, repita recusando.

Saídas esperadas

- trace do grafo com eventos (agente, ferramentas, agente)
- memória reaproveitada na mesma thread_id
- contexto isolado com thread_id nova
- pausa antes da ferramenta (HITL)
- retomada após aprovação e bloqueio após recusa

Exemplo de execução

```
# 1. executar em modo de streaming
for ev in app.stream({"messages": [{"role": "user", "content": "qual é o saldo?"}]},
    config={"thread_id": "t1"}, stream_mode="updates"):
    print(ev)

{'agente': {...}}
{'ferramentas': {...}}
{'agente': {...}}
```

08

Streaming, testes e fechamento

Mostrar progresso, e provar que o roteamento está certo.

Quatro modos, quatro usos

stream_mode	O que entrega	Quando usar
values	O estado completo depois de cada nó	Depuração: ver o estado inteiro evoluindo
updates	Só o delta, com o nome do nó	Log de progresso e traço em disco
messages	Token a token, como um chat	Interface para o usuário final
custom	O que você emitir de dentro do nó	Barras de progresso e eventos próprios

Streaming do grafo: observar a execução passo a passo

Sequência de execução (streaming de updates)



Eventos recebidos no stream (exemplo)

```
> {"agente": {"mensagens": ["Vou consultar a base de conhecimentos..."]}}
> {"ferramentas": {"tool_calls": [{"name": "buscar_docs", "args": {...}]}}
> {"agente": {"mensagens": ["Com base no resultado, a resposta é ..."]}}
```

Exemplo de código

```
1 for evento in app.stream(entrada, cfg, stream_mode="updates"):
2     print(evento)
```

- O streaming revela, em tempo real, a sequência de eventos entre os nós (agente → ferramentas → agente).
- Ajuda a depurar, pois permite ver exatamente o que cada nó produz a cada passo.
- É uma evidência prática de que o grafo realmente está em ciclo, com várias iterações até concluir a tarefa.

Toda lógica de roteamento deve ser testável sem modelo

```
def test_rotear_termina_sem_toolcall():
    estado = {"messages": [AIMessage(content="pronto")], "passos": 1}
    assert rotear(estado) == END

def test_rotear_corta_no_limite():
    ai = AIMessage(content="", tool_calls=[
        {"name": "calcular", "args": {"expressao": "1+1"}, "id": "x"}])
    assert rotear({"messages": [ai], "passos": 10}) == END

def test_politica_negativa_fora_da_pasta():
    assert proteger_fora_da_pasta(
        "salvar_relatorio", {"caminho": "/etc/passwd"}) is not None

# nenhum destes testes chama o LLM. São rápidos e determinísticos.
```

Se o seu roteador precisa do LLM para ser testado, ele está fazendo coisa demais. Separe a decisão, que é testável, da geração, que não é determinística.

O NEXUS ao fim do dia 3

2

nós: agente e ferramentas

6

ferramentas registradas

2

políticas de PreToolUse

1

banco SQLite de checkpoints

O limite de passos agora é DADO, e não variável de loop: dá para inspecioná-lo, persisti-lo e mostrá-lo na interface. Essa mudança de lugar, de código para dado, é o que o grafo compra.

Erros clássicos do dia 3

Sintoma	Causa	Correção
Histórico some a cada nó	Falta o reducer add_messages	Adicione Annotated com add_messages
KeyError numa chave do estado	O nó devolveu o estado inteiro	Devolva só o delta
Grafo não para nunca	A condicional nunca retorna END	Contador de passos no estado
Memória não persiste	Sem thread_id no config	configurable com thread_id
SQLite: database is locked	Vários processos no mesmo arquivo	Um saver por processo
invoke(None) reinicia tudo	Faltou o checkpointer	Compile com checkpointer
Duas conversas se misturam	Mesmo thread_id	Um thread_id por conversa, com o dono junto

As cinco camadas, e o que fizemos hoje

Camada	O que construímos hoje	Onde aprofunda
Instructions	Ordem de preferência entre ferramentas, no prompt	Dia 4, um prompt por especialista
Constraints	Hooks que negam, sandbox de caminho, aprovação humana	Dia 4, orçamento por agente
Feedback	Recusa que explica, testes de roteamento e política	Dia 4, com o Crítico
Memory	Checkpointing, thread_id com dono, poda de contexto	Dia 4, memória entre agentes
Orchestration	Um grafo, dois nós; a estrutura já está pronta	Dia 4, cinco topologias

PARA AMANHÃ

Um agente com nove ferramentas e um prompt de sistema de sessenta linhas começa a errar.

Escolhe a ferramenta errada. Esquece regras do começo do prompt. Mistura os critérios de duas tarefas diferentes. O que acontece se, em vez de um agente com nove ferramentas, tivermos três agentes com três ferramentas cada?